Git By Example

Dariush Abbasi

Table of Contents

Part I — Foundations

1. Getting Started
2. The Basics
3. Viewing History
4. Undoing Things

Part II — Branching & Collaboration

5. Branches
6. Merging
7. Rebasing
8. Remotes

Part III — Intermediate Techniques

9. Stashing
10. Tagging
11. Cherry Picking
12. Interactive Rebase

Part IV — Advanced Topics

13. Git Internals
14. Reflog
15. Bisect
16. Worktrees
17. Submodules

Part V — Real-World Workflows

18. Branching Strategies
19. Commit Messages
20. Git Hooks
21. Tips And Tricks

Appendices

Config Cheatsheet
Gitignore Patterns
Aliases

Part I

Foundations

Chapter 1

Getting Started

Installing Git

macOS

```bash
# Option 1: Xcode Command Line Tools (easiest)
xcode-select --install

# Option 2: Homebrew
brew install git
```

Linux (Debian/Ubuntu)

```bash
sudo apt update
sudo apt install git
```

Linux (Fedora)

```bash
sudo dnf install git
```

Windows

Download the installer from git-scm.com, or use:

```powershell
winget install Git.Git
```

Verify Installation

```bash
git --version
```

```
git version 2.44.0
```

Configuring Git

Before your first commit, tell Git who you are.

Example: Set your identity

```bash
git config --global user.name "Dariush Abbasi"
git config --global user.email "dariush@example.com"
```

> **What happened?**
> Git wrote these values to `~/.gitconfig`. Every commit you make will be stamped with this name and email.

Example: See your config

```bash
git config --list
```

```
user.name=Dariush Abbasi
user.email=dariush@example.com
```

Example: Set your default editor

```bash
git config --global core.editor "vim"
```

Other popular choices: `"code --wait"` (VS Code), `"nano"`, `"subl -n -w"` (Sublime Text).

Example: Set default branch name

```bash
git config --global init.defaultBranch main
```

> **What happened?**
> New repositories you create with `git init` will use `main` instead of `master` as the default branch name.

Creating Your First Repository

Example: Initialize a new repo

```bash
mkdir my-project
cd my-project
git init
```

```
Initialized empty Git repository in /home/you/my-project/.git/
```

Example: See what Git created

```bash
ls -la .git/
```

```
drwxr-xr-x  HEAD
drwxr-xr-x  config
drwxr-xr-x  hooks/
drwxr-xr-x  objects/
drwxr-xr-x  refs/
```

What happened?

Git created a hidden `.git/` directory. This single folder *is* your repository — all history, branches, and configuration live inside it. Your working directory is just a checkout of one version.

Cloning an Existing Repository

Example: Clone a repo from GitHub

```bash
git clone https://github.com/boringcollege/git-by-example.git
cd git-by-example
```

```
Cloning into 'git-by-example'...
remote: Enumerating objects: 42, done.
remote: Total 42 (delta 0), reused 0 (delta 0)
Receiving objects: 100% (42/42), done.
```

What happened?

Git downloaded the entire repository — all files, all branches, all history — to your machine. You now have a full, independent copy.

Chapter 2

The Basics: Add, Commit, Status

The Three Areas

Git has three areas you need to understand:

```
Working Directory  →  Staging Area (Index)  →  Repository (.git)
    (your files)        (git add)               (git commit)
```

- **Working Directory**: the files you see and edit.
- **Staging Area**: a draft of your next commit. You choose what goes in.
- **Repository**: the permanent history.

Checking Status

Example: Status of a fresh repo

```bash
git status
```

```
On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

Example: Create a file and check status

```bash
echo "# My Project" > README.md
git status
```

```
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md

nothing added to commit but untracked files present (use "git add" to track)
```

What happened?

Git sees a new file it hasn't been told to track. It's "untracked."

Adding Files to the Staging Area

Example: Stage a single file

```bash
git add README.md
git status
```

```
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   README.md
```

Example: Stage everything

```bash
echo "console.log('hello');" > index.js
echo "node_modules/" > .gitignore
git add .
git status
```

```
Changes to be committed:
        new file:   .gitignore
        new file:   README.md
        new file:   index.js
```

What happened?

`git add .` stages all new and modified files in the current directory and below.

Example: Stage only part of a file

```bash
git add -p index.js
```

Git will show you each change (hunk) and ask:

```
Stage this hunk [y,n,q,a,d,s,e,?]?
```

Type `y` to stage that hunk, `n` to skip it. This lets you create precise, focused commits even when you've made many changes to one file.

Making a Commit

Example: Commit with a message

```bash
git commit -m "Initial commit: add README, index.js, and .gitignore"
```

```
[main (root-commit) a1b2c3d] Initial commit: add README, index.js, and .gitignore
 3 files changed, 3 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 README.md
 create mode 100644 index.js
```

Example: Commit with a longer message

```bash
git commit
```

This opens your editor. Write:

```
Add user authentication module

- Add login/logout endpoints
- Add session middleware
- Add password hashing with bcrypt
```

> **What happened?**
> Git took a snapshot of everything in the staging area and stored it permanently. The commit gets a unique SHA hash (`a1b2c3d`).

The Edit-Stage-Commit Cycle

Here's the workflow you'll use hundreds of times a day:

```bash
# 1. Edit files
echo "function greet() { return 'hi'; }" >> index.js

# 2. See what changed
git status
git diff

# 3. Stage the changes
git add index.js

# 4. Commit
git commit -m "Add greet function"
```

Viewing Differences

Example: See unstaged changes

```bash
echo "// TODO: add tests" >> index.js
git diff
```

```diff
diff --git a/index.js b/index.js
index 1a2b3c4..5d6e7f8 100644
--- a/index.js
+++ b/index.js
@@ -1,2 +1,3 @@
 console.log('hello');
 function greet() { return 'hi'; }
+// TODO: add tests
```

Example: See staged changes (what will be in the next commit)

```bash
git add index.js
git diff --staged
```

```diff
+// TODO: add tests
```

Example: See both staged and unstaged

```bash
git diff HEAD
```

Removing and Renaming Files

Example: Remove a file

```bash
git rm old-file.js
git commit -m "Remove old-file.js"
```

Example: Rename a file

```bash
git mv index.js app.js
git status
```

```
Changes to be committed:
        renamed:    index.js -> app.js
```

What happened?

`git mv` is a shortcut for `mv` + `git rm` + `git add`. Git detects renames by comparing file content.

Chapter 3

Viewing History

The Log

Example: Basic log

```bash
git log
```

```
commit e4f5a6b (HEAD -> main)
Author: Dariush Abbasi <dariush@example.com>
Date:   Mon Mar 10 14:32:00 2025 +0330

    Add greet function

commit a1b2c3d
Author: Dariush Abbasi <dariush@example.com>
Date:   Mon Mar 10 14:30:00 2025 +0330

    Initial commit: add README, index.js, and .gitignore
```

Example: One-line log

```bash
git log --oneline
```

```
e4f5a6b (HEAD -> main) Add greet function
a1b2c3d Initial commit: add README, index.js, and .gitignore
```

Example: Log with a graph

```bash
git log --oneline --graph --all
```

```
* f7g8h9i (feature/login) Add login page
| * e4f5a6b (HEAD -> main) Add greet function
|/
* a1b2c3d Initial commit
```

What happened?

`--graph` draws the branch structure as ASCII art. `--all` shows all branches, not just the one you're on.

Example: Log for a specific file

```bash
git log --oneline -- README.md
```

```
a1b2c3d Initial commit: add README, index.js, and .gitignore
```

Example: Search commits by message

```bash
git log --oneline --grep="fix"
```

Example: Search commits by code change

```bash
git log --oneline -S "greet"
```

What happened?

`-S` (the "pickaxe") finds commits where the number of occurrences of "greet" changed — i.e., where that string was added or removed.

Show a Specific Commit

Example: Show full commit details

```bash
git show e4f5a6b
```

```
commit e4f5a6b
Author: Dariush Abbasi <dariush@example.com>
Date:   Mon Mar 10 14:32:00 2025 +0330

    Add greet function

diff --git a/index.js b/index.js
index 1a2b3c4..5d6e7f8 100644
--- a/index.js
+++ b/index.js
@@ -1 +1,2 @@
 console.log('hello');
+function greet() { return 'hi'; }
```

Example: Show only the files that changed

```bash
git show --stat e4f5a6b
```

```
 index.js | 1 +
 1 file changed, 1 insertion(+)
```

Blame: Who Changed What

Example: See who wrote each line

```bash
git blame index.js
```

```
a1b2c3d (Dariush Abbasi 2025-03-10 14:30:00 +0330 1) console.log('hello');
e4f5a6b (Dariush Abbasi 2025-03-10 14:32:00 +0330 2) function greet() { return 'hi'; }
```

What happened?

`git blame` annotates each line with the commit, author, and date of the last change. Despite the dramatic name, it's a tool for understanding history, not assigning fault.

Shortlog: Contributor Summary

Example: See commits grouped by author

```bash
git shortlog -sn
```

```
    15  Dariush Abbasi
     3  Jane Doe
     1  Bob Smith
```

Comparing Points in Time

Example: Diff between two commits

```bash
git diff a1b2c3d..e4f5a6b
```

Example: Diff between two branches

```bash
git diff main..feature/login
```

Example: What changed in the last 3 commits?

```bash
git diff HEAD~3..HEAD
```

Example: Just show file names that changed

```bash
git diff --name-only HEAD~3..HEAD
```

```
index.js
README.md
```

Chapter 4

Undoing Things

Git gives you many ways to undo mistakes. The right tool depends on *where* the mistake is.

Undo Cheatsheet

```
Mistake is in...            Use...
_____________________________________
Working directory           git checkout -- <file>  or  git restore <file>
Staging area                git reset HEAD <file>   or  git restore --staged <file>
Last commit (not pushed)    git commit --amend
Older commit (not pushed)   git rebase -i
Any commit (already pushed) git revert
```

Discard Changes in Working Directory

Example: Throw away uncommitted edits

BASH

```bash
echo "BROKEN CODE" > index.js
git status
```

```
Changes not staged for commit:
        modified:   index.js
```

BASH

```bash
git restore index.js
cat index.js
```

```
console.log('hello');
function greet() { return 'hi'; }
```

What happened?

`git restore` replaced the working directory version with the version from the staging area (which matches the last commit). Your changes are gone — this is not reversible.

Unstage a File

Example: Remove a file from the staging area

```bash
echo "secret=abc123" > .env
git add .env        # Oops, didn't mean to stage this
git restore --staged .env
git status
```

```
Untracked files:
        .env
```

What happened?

The file is still on disk, but it's no longer staged for commit. Your changes are safe.

Amend the Last Commit

Example: Fix a typo in the last commit message

```bash
git commit --amend -m "Add greet function to index.js"
```

Example: Add a forgotten file to the last commit

```bash
echo "test" > test.js
git add test.js
git commit --amend --no-edit
```

What happened?

`--amend` replaces the last commit with a new one. The old commit is discarded. **Never amend commits you've already pushed** — it rewrites history.

Revert a Commit

Example: Undo a commit that's already been pushed

```bash
git log --oneline
```

```
c3d4e5f Add broken feature
e4f5a6b Add greet function
a1b2c3d Initial commit
```

```bash
git revert c3d4e5f
```

```
[main f6g7h8i] Revert "Add broken feature"
 1 file changed, 1 deletion(-)
```

```bash
git log --oneline
```

```
f6g7h8i Revert "Add broken feature"
c3d4e5f Add broken feature
e4f5a6b Add greet function
a1b2c3d Initial commit
```

What happened?

`git revert` creates a *new* commit that undoes the changes from the target commit. History is preserved. This is the safe way to undo public commits.

Reset: Moving the Branch Pointer

Reset is powerful and comes in three flavors:

```
                    Moves HEAD?  Resets Index?  Resets Working Dir?
git reset --soft                   X               X
git reset --mixed                                  X
git reset --hard
```

Example: Soft reset — undo commit, keep changes staged

```bash
git reset --soft HEAD~1
git status
```

```
Changes to be committed:
        modified:   index.js
```

What happened?

The commit is gone, but your changes are still staged. Useful when you want to re-do a commit.

Example: Mixed reset (default) — undo commit, unstage changes

```bash
git reset HEAD~1
git status
```

```
Changes not staged for commit:
        modified:   index.js
```

Example: Hard reset — undo everything

```bash
git reset --hard HEAD~1
```

> **Warning**
> `--hard` permanently discards uncommitted changes. Use with extreme care.

Example: Reset to a specific commit

```bash
git reset --hard a1b2c3d
```

> **What happened?**
> Your branch now points to `a1b2c3d`. All commits after it are orphaned (but recoverable via `git reflog` for ~30 days).

Clean: Remove Untracked Files

Example: See what would be deleted

```bash
git clean -n
```

```
Would remove temp.txt
Would remove debug.log
```

Example: Actually delete them

```bash
git clean -f
```

Example: Remove untracked directories too

```bash
git clean -fd
```

Part II

Branching & Collaboration

Chapter 5

Branches

A branch in Git is just a pointer to a commit. Creating a branch is instant and cheap — there's no reason not to branch early and often.

Listing Branches

Example: See local branches

```bash
git branch
```

```
  feature/login
* main
```

The `*` marks the branch you're currently on.

Example: See all branches (including remote)

```bash
git branch -a
```

```
  feature/login
* main
  remotes/origin/main
  remotes/origin/feature/login
```

Creating a Branch

Example: Create a branch

```bash
git branch feature/signup
```

Example: Create and switch in one command

```bash
git switch -c feature/signup
```

Or the older syntax:

```bash
git checkout -b feature/signup
```

> **What happened?**
>
> Git created a new pointer called `feature/signup` pointing at the same commit you're currently on. `switch -c` also moves `HEAD` to that branch.

Switching Branches

Example: Switch to an existing branch

```bash
git switch main
```

```
Switched to branch 'main'
```

Example: What happens to uncommitted changes?

```bash
# On feature/signup
echo "signup code" > signup.js
git switch main
```

```
error: Your local changes to the following files would be overwritten by checkout:
        signup.js
Please commit your changes or stash them before you switch branches.
```

> **What happened?**
>
> Git protects you from losing work. You have three options: commit, stash (see Chapter 9), or discard.

Renaming a Branch

Example: Rename the current branch

```bash
git branch -m new-name
```

Example: Rename any branch

```bash
git branch -m old-name new-name
```

Deleting a Branch

Example: Delete a merged branch

```bash
git branch -d feature/signup
```

```
Deleted branch feature/signup (was e4f5a6b).
```

Example: Force-delete an unmerged branch

```bash
git branch -D feature/experiment
```

> **Warning**
> `-D` deletes the branch even if its changes haven't been merged anywhere. The commits become orphaned.

Understanding HEAD

`HEAD` is a pointer to "where you are right now." Normally, it points to a branch, which points to a commit.

```
HEAD → main → e4f5a6b
```

Example: See where HEAD points

```bash
cat .git/HEAD
```

```
ref: refs/heads/main
```

Example: Detached HEAD (looking at an old commit)

```bash
git checkout a1b2c3d
```

```
Note: switching to 'a1b2c3d'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.
```

What happened?

HEAD now points directly to a commit instead of a branch. You can look around and experiment, but any new commits won't belong to any branch. Switch back with `git switch main`.

Chapter 6

Merging

Merging takes the work from one branch and integrates it into another.

Fast-Forward Merge

When the target branch hasn't diverged, Git simply moves the pointer forward.

Example: Fast-forward merge

```bash
git switch main
git merge feature/signup
```

```
Updating a1b2c3d..d5e6f7g
Fast-forward
 signup.js | 5 +++++
 1 file changed, 5 insertions(+)
 create mode 100644 signup.js
```

Before:

```
main:           A --- B
                         \
feature/signup:           C --- D
```

After:

```
main:           A --- B --- C --- D
```

> **What happened?**
>
> No merge commit was created. Git simply moved `main` forward to where `feature/signup` was.

Three-Way Merge

When both branches have new commits, Git creates a merge commit.

Example: Merge with a merge commit

```bash
# Set up: both branches have diverged
git switch main
echo "main work" >> app.js && git add . && git commit -m "Work on main"

git switch feature/login
echo "login work" >> login.js && git add . && git commit -m "Work on login"

git switch main
git merge feature/login
```

```
Merge made by the 'ort' strategy.
 login.js | 1 +
 1 file changed, 1 insertion(+)
```

```bash
git log --oneline --graph
```

```
*   h8i9j0k Merge branch 'feature/login'
|\
| * f7g8h9i Work on login
* | e4f5a6b Work on main
|/
* a1b2c3d Initial commit
```

Handling Merge Conflicts

Conflicts happen when both branches changed the same lines.

Example: Create a conflict

```bash
# On main
echo "version = 2" > config.txt && git add . && git commit -m "Set version 2"

# On feature branch
git switch -c feature/config
echo "version = 3" > config.txt && git add . && git commit -m "Set version 3"

# Try to merge
git switch main
git merge feature/config
```

```
Auto-merging config.txt
CONFLICT (content): Merge conflict in config.txt
Automatic merge failed; fix conflicts and then commit the result.
```

Example: See the conflict markers

```bash
cat config.txt
```

```
<<<<<<< HEAD
version = 2
=======
version = 3
>>>>>>> feature/config
```

Example: Resolve the conflict

Edit the file to keep what you want:

```
version = 3
```

Then:

```bash
git add config.txt
git commit -m "Merge feature/config: use version 3"
```

> **What happened?**
>
> Git couldn't automatically decide which version to keep, so it put both versions in the file with markers. You manually resolved it, then committed the result.

Example: Abort a merge

If you're in the middle of a conflict and want to start over:

```bash
git merge --abort
```

Everything goes back to the state before you ran `git merge`.

Merge Strategies

Example: Force a merge commit (no fast-forward)

```bash
git merge --no-ff feature/signup
```

This creates a merge commit even when fast-forward is possible. Useful for preserving the fact that work happened on a branch.

Example: Squash merge

```bash
git merge --squash feature/signup
git commit -m "Add signup feature"
```

What happened?

All commits from the branch are combined into a single set of changes in the staging area. You then commit them as one commit. The branch's individual commits don't appear in `main`'s history.

Chapter 7

Rebasing

Rebasing replays your commits on top of another branch, creating a linear history.

Merge vs. Rebase

```
MERGE creates this:              REBASE creates this:

*   Merge branch 'feature'       * C' (feature, rebased)
|\                                * B' (feature, rebased)
| * C (feature)                   * E (main)
| * B (feature)                   * D (main)
* | E (main)                      * A
* | D (main)
|/
* A
```

Both integrate the same changes. Rebase gives you a cleaner, linear history.

Basic Rebase

Example: Rebase a feature branch onto main

```bash
git switch feature/signup
git rebase main
```

```
Successfully rebased and updated refs/heads/feature/signup.
```

```bash
git log --oneline
```

```
b2c3d4e (HEAD -> feature/signup) Add signup form
a9b8c7d (main) Latest main commit
```

What happened?

Git took the commits from `feature/signup` that aren't in `main`, then replayed them one by one on top of `main`. Each replayed commit gets a new hash because its parent changed.

Example: Then fast-forward main

```bash
git switch main
git merge feature/signup
```

```
Fast-forward
```

Now `main` has a clean, linear history with no merge commit.

Handling Rebase Conflicts

Conflicts work similarly to merge conflicts, but you resolve them commit-by-commit.

Example: Conflict during rebase

```bash
git switch feature/config
git rebase main
```

```
CONFLICT (content): Merge conflict in config.txt
error: could not apply b2c3d4e... Update config
hint: Resolve all conflicts manually, then run "git rebase --continue".
hint: To abort, run "git rebase --abort".
```

```bash
# 1. Fix the conflict in config.txt
# 2. Stage the fix
git add config.txt
# 3. Continue the rebase
git rebase --continue
```

Example: Abort a rebase

```bash
git rebase --abort
```

Everything goes back to how it was before the rebase started.

Example: Skip a conflicting commit

```bash
git rebase --skip
```

> **Warning**
> This drops the conflicting commit entirely.

The Golden Rule of Rebasing

> ***Never rebase commits that have been pushed and shared with others.***

Rebasing rewrites commit hashes. If someone else has based work on the original commits, rebasing will cause chaos.

```
Safe to rebase:
  Your local commits that haven't been pushed yet.

NOT safe to rebase:
  Anything on a shared branch that others have pulled.
```

Rebase onto a Specific Commit

Example: Rebase onto a specific point

```bash
git rebase --onto main feature/old-base feature/new-feature
```

What happened?

This takes the commits between `feature/old-base` and `feature/new-feature`, and replays them on top of `main`. Useful when you branched from the wrong point.

Chapter 8

Working with Remotes

A remote is a copy of your repository hosted somewhere else — usually GitHub, GitLab, or Bitbucket.

Listing Remotes

Example: See configured remotes

```bash
git remote -v
```

```
origin  https://github.com/boringcollege/git-by-example.git (fetch)
origin  https://github.com/boringcollege/git-by-example.git (push)
```

What happened?

`origin` is the default name for the remote you cloned from. You can have multiple remotes.

Adding and Removing Remotes

Example: Add a remote

```bash
git remote add upstream https://github.com/original-author/project.git
```

Example: Remove a remote

```bash
git remote remove upstream
```

Example: Rename a remote

```bash
git remote rename origin github
```

Fetching

Fetch downloads new data from the remote but does **not** change your working files.

Example: Fetch from origin

```bash
git fetch origin
```

```
remote: Enumerating objects: 5, done.
remote: Total 3 (delta 1), reused 3 (delta 1)
Unpacking objects: 100% (3/3), done.
From https://github.com/boringcollege/git-by-example
   a1b2c3d..d4e5f6g  main       -> origin/main
```

Example: See what's new

```bash
git log main..origin/main --oneline
```

```
d4e5f6g Fix typo in README
c3d4e5f Add contributing guidelines
```

What happened?

`git fetch` updated your remote-tracking branches (`origin/main`) without touching your local `main`. You can inspect the changes before deciding to integrate them.

Pulling

Pull = fetch + merge (or fetch + rebase).

Example: Pull with merge (default)

```bash
git pull origin main
```

Example: Pull with rebase (cleaner history)

```bash
git pull --rebase origin main
```

Example: Set rebase as default for pull

```bash
git config --global pull.rebase true
```

Pushing

Example: Push to the remote

```bash
git push origin main
```

Example: Push a new branch

```bash
git switch -c feature/dark-mode
# ... make commits ...
git push -u origin feature/dark-mode
```

```
Branch 'feature/dark-mode' set up to track remote branch 'feature/dark-mode' from 'origin'.
```

What happened?

`-u` (or `--set-upstream`) links your local branch to the remote branch. After this, you can just run `git push` without specifying the remote and branch.

Example: Push rejected (remote has new commits)

```bash
git push origin main
```

```
! [rejected]        main -> main (fetch first)
error: failed to push some refs
hint: Updates were rejected because the remote contains work that you do not have locally.
```

Fix: pull first, then push.

```bash
git pull --rebase origin main
git push origin main
```

Example: Force push (after rebase)

```bash
git push --force-with-lease origin feature/my-branch
```

What happened?

`--force-with-lease` is a safer force push. It refuses to overwrite the remote if someone else has pushed commits you haven't seen. Use it instead of `--force`.

Tracking Branches

Example: See which local branches track which remote branches

```bash
git branch -vv
```

```
  feature/login  f7g8h9i [origin/feature/login] Add login page
* main           e4f5a6b [origin/main] Add greet function
```

Example: Set up tracking for an existing branch

```bash
git branch --set-upstream-to=origin/main main
```

Part III

Intermediate Techniques

Chapter 9

Stashing

Stashing saves your uncommitted changes temporarily so you can work on something else, then come back.

Basic Stash

Example: Stash your changes

```bash
# You're in the middle of something
echo "work in progress" >> app.js

git stash
```

```
Saved working directory and index state WIP on main: e4f5a6b Add greet function
```

```bash
git status
```

```
On branch main
nothing to commit, working tree clean
```

What happened?

Git saved your changes to a stack and restored your working directory to a clean state. You can now switch branches safely.

Example: Bring your changes back

```bash
git stash pop
```

```
On branch main
Changes not staged for commit:
        modified:   app.js

Dropped refs/stash@{0}
```

What happened?

pop applies the most recent stash and removes it from the stack.

Stash with a Message

Example: Name your stash

```bash
git stash push -m "WIP: dark mode CSS"
```

```
Saved working directory and index state On main: WIP: dark mode CSS
```

Listing Stashes

Example: See all stashes

```bash
git stash list
```

```
stash@{0}: On main: WIP: dark mode CSS
stash@{1}: WIP on main: e4f5a6b Add greet function
```

Applying Without Removing

Example: Apply but keep in stack

```bash
git stash apply stash@{1}
```

What happened?

Unlike `pop`, `apply` restores the changes but keeps the stash entry. Useful if you want to apply the same stash to multiple branches.

Stashing Untracked Files

Example: Include untracked files

```bash
echo "new file" > new.js
git stash push -u -m "Include new files"
```

By default, `git stash` only saves tracked files. `-u` includes untracked files too.

Example: Include everything (even ignored files)

```bash
git stash push -a -m "Include absolutely everything"
```

Dropping a Stash

Example: Remove a specific stash

```bash
git stash drop stash@{1}
```

Example: Remove all stashes

```bash
git stash clear
```

Creating a Branch from a Stash

Example: Stash → branch

```bash
git stash branch feature/dark-mode stash@{0}
```

> **What happened?**
> Git created a new branch from the commit where you originally stashed, applied the stash, and dropped it. Useful when you realize your stashed work should live on its own branch.

Chapter 10

Tagging

Tags mark specific points in history — usually releases.

Types of Tags

Git has two kinds of tags:

- **Lightweight**: just a pointer to a commit (like a branch that doesn't move).
- **Annotated**: a full Git object with a tagger name, date, message, and optional GPG signature.

Use annotated tags for releases. Use lightweight tags for personal bookmarks.

Creating Tags

Example: Annotated tag

```bash
git tag -a v1.0.0 -m "Release version 1.0.0"
```

Example: Lightweight tag

```bash
git tag v1.0.0-beta
```

Example: Tag an older commit

```bash
git tag -a v0.9.0 -m "Retroactive tag for beta" a1b2c3d
```

Listing Tags

Example: List all tags

```bash
git tag
```

```
v0.9.0
v1.0.0
v1.0.0-beta
```

Example: Filter tags

```bash
git tag -l "v1.*"
```

```
v1.0.0
v1.0.0-beta
```

Example: See tag details

```bash
git show v1.0.0
```

```
tag v1.0.0
Tagger: Dariush Abbasi <dariush@example.com>
Date:   Mon Mar 10 15:00:00 2025 +0330

Release version 1.0.0

commit e4f5a6b...
```

Pushing Tags

Tags are **not** pushed by default.

Example: Push a single tag

```bash
git push origin v1.0.0
```

Example: Push all tags

```bash
git push origin --tags
```

Deleting Tags

Example: Delete a local tag

```bash
git tag -d v1.0.0-beta
```

Example: Delete a remote tag

```bash
git push origin --delete v1.0.0-beta
```

Checking Out a Tag

Example: Look at a tagged version

```bash
git checkout v1.0.0
```

```
Note: switching to 'v1.0.0'.
You are in 'detached HEAD' state.
```

What happened?

You can look at the code as it was at `v1.0.0`, but you're in detached HEAD state. To make changes from here, create a branch: `git switch -c hotfix/v1.0.1`.

Chapter 11

Cherry-Picking

Cherry-pick copies a single commit from one branch to another.

When to Cherry-Pick

Cherry-picking is useful when you need one specific commit from another branch without merging the whole branch. Common scenarios: applying a hotfix from `main` to a release branch, or grabbing a single bug fix from a feature branch.

Basic Cherry-Pick

Example: Apply a single commit

```bash
git switch main
git cherry-pick f7g8h9i
```

```
[main k1l2m3n] Fix login redirect bug
 Date: Mon Mar 10 16:00:00 2025 +0330
 1 file changed, 2 insertions(+), 1 deletion(-)
```

What happened?

Git created a *new* commit on `main` with the same changes as `f7g8h9i`, but with a different hash. The original commit on the other branch is untouched.

Cherry-Pick Multiple Commits

Example: Pick a range

```bash
git cherry-pick a1b2c3d..e4f5a6b
```

> **Warning**
> The range is **exclusive** of the first commit. This picks everything *after* `a1b2c3d` up to and including `e4f5a6b`.

Example: Pick several specific commits

```bash
git cherry-pick a1b2c3d e4f5a6b f7g8h9i
```

Handling Conflicts

Example: Conflict during cherry-pick

```bash
git cherry-pick f7g8h9i
```

```
CONFLICT (content): Merge conflict in app.js
error: could not apply f7g8h9i... Fix login redirect bug
```

```bash
# Fix the conflict in app.js
git add app.js
git cherry-pick --continue
```

Example: Abort the cherry-pick

```bash
git cherry-pick --abort
```

Cherry-Pick Without Committing

Example: Apply changes but don't commit

```bash
git cherry-pick --no-commit f7g8h9i
git status
```

```
Changes to be committed:
        modified:   app.js
```

What happened?

The changes are staged but not committed. You can modify them further or combine them with other changes before committing.

Chapter 12

Interactive Rebase

Interactive rebase lets you rewrite history: reorder, squash, edit, or drop commits before sharing them.

Starting an Interactive Rebase

Example: Rebase the last 4 commits

```bash
git rebase -i HEAD~4
```

Your editor opens with:

```
pick a1b2c3d Add user model
pick b2c3d4e Add user controller
pick c3d4e5f Fix typo in user model
pick d4e5f6g Add user routes
```

The Commands

```
pick   = use commit as-is
reword = use commit, but edit the message
edit   = use commit, but stop to amend it
squash = meld into previous commit (keep message)
fixup  = meld into previous commit (discard message)
drop   = remove commit entirely
```

Squash: Combine Commits

Example: Squash a typo fix into the original commit

Change the file to:

```
pick a1b2c3d Add user model
fixup c3d4e5f Fix typo in user model
pick b2c3d4e Add user controller
pick d4e5f6g Add user routes
```

What happened?

We moved the typo fix right after the original commit and changed `pick` to `fixup`. Git will combine them into one commit and discard the "Fix typo" message.

```bash
git log --oneline
```

```
g7h8i9j Add user routes
f6g7h8i Add user controller
e5f6g7h Add user model
```

Three clean commits instead of four.

Reword: Fix a Commit Message

Example: Change a message

```
reword a1b2c3d Add user model
pick b2c3d4e Add user controller
```

Save and close. Git will open a new editor for the commit message of `a1b2c3d`.

Edit: Modify a Past Commit

Example: Add a forgotten file to an old commit

```
edit a1b2c3d Add user model
pick b2c3d4e Add user controller
```

Save and close. Git stops at that commit:

```bash
# Add the forgotten file
git add user_model_test.js
git commit --amend --no-edit
git rebase --continue
```

Reorder Commits

Example: Change the order

Simply rearrange the lines:

```
pick d4e5f6g Add user routes
pick a1b2c3d Add user model
pick b2c3d4e Add user controller
```

> **Warning**
> Reordering can cause conflicts if commits depend on each other.

Drop: Remove a Commit

Example: Delete a commit from history

```
pick a1b2c3d Add user model
drop b2c3d4e Add experimental debug logging
pick c3d4e5f Add user controller
```

Or simply delete the line entirely — same effect.

Autosquash

If you name fixup commits with `fixup!` or `squash!` prefix, Git can organize them automatically.

Example: Autosquash workflow

```bash
# Original commit
git commit -m "Add user model"

# Later, a fix for that commit
git commit -m "fixup! Add user model"

# Rebase with autosquash
git rebase -i --autosquash HEAD~3
```

Git automatically puts the fixup commit right after its target and marks it as `fixup`.

Part IV

Advanced Topics

Chapter 13

Git Internals: Objects & Refs

Understanding Git's internals makes everything else click. Git is, at its core, a content-addressable filesystem with a version control system built on top.

The Four Object Types

Everything in Git is stored as one of four types of objects:

```
blob    → file contents (no filename, just data)
tree    → directory listing (filenames + pointers to blobs/trees)
commit  → snapshot (pointer to tree + author + message + parent)
tag     → annotated tag (pointer to commit + tagger + message)
```

Exploring Objects

Example: See the type of an object

```bash
git cat-file -t HEAD
```

```
commit
```

Example: See the content of a commit

```bash
git cat-file -p HEAD
```

```
tree 4b825dc642cb6eb9a060e54bf899d15d2cb1b5d6
parent a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0
author Dariush Abbasi <dariush@example.com> 1710072720 +0330
committer Dariush Abbasi <dariush@example.com> 1710072720 +0330

Add greet function
```

Example: See a tree object

```bash
git cat-file -p HEAD^{tree}
```

```
100644 blob a906cb2a4a904a152e80877d4088654daad0c859    .gitignore
100644 blob 1f7a7a472abf3dd9643fd615f6da379c4acb3e3a    README.md
100644 blob 5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1b2c3d4e    index.js
```

Example: See a blob (file contents)

```bash
git cat-file -p 5d6e7f8
```

```
console.log('hello');
function greet() { return 'hi'; }
```

> **What happened?**
> A blob stores raw file content. It has no filename — the tree object maps filenames to blobs.

How SHA Hashes Work

Every object is identified by a SHA-1 hash of its content.

Example: Hash a file manually

```bash
echo -n "hello" | git hash-object --stdin
```

```
ce013625030ba8dba906f756967f9e9ca394464a
```

Example: Store an object

```bash
echo "hello world" | git hash-object -w --stdin
```

```
95d09f2b10159347eece71399a7e2e907ea3df4f
```

> **What happened?**
> `-w` actually writes the object to `.git/objects/`. The hash is derived from the content, so identical content always produces the same hash.

Where Objects Live

```bash
find .git/objects -type f | head -5
```

```
.git/objects/95/d09f2b10159347eece71399a7e2e907ea3df4f
.git/objects/ce/013625030ba8dba906f756967f9e9ca394464a
.git/objects/a9/06cb2a4a904a152e80877d4088654daad0c859
```

Objects are stored in directories named by the first two characters of their hash.

Refs: Named Pointers

Branches and tags are just files containing a commit hash.

Example: See what a branch points to

```bash
cat .git/refs/heads/main
```

```
e4f5a6b7c8d9e0f1a2b3c4d5e6f7a8b9c0d1e2f3
```

Example: See what HEAD points to

```bash
cat .git/HEAD
```

```
ref: refs/heads/main
```

> **What happened?**
>
> A branch is literally a 41-byte file (40 hex chars + newline). HEAD is a symbolic ref that usually points to a branch. That's why branching in Git is so fast — there's nothing to copy.

The Commit Graph

Commits form a directed acyclic graph (DAG). Each commit points to its parent(s):

```
a1b2c3d  ←  e4f5a6b  ←  f7g8h9i  ←  h8i9j0k
  │                                    │
  └──────────────── g6h7i8j ───────────┘
                 (second parent of merge commit)
```

Understanding this graph is the key to understanding `merge`, `rebase`, `reset`, and `cherry-pick`.

Chapter 14

Reflog: Your Safety Net

The reflog records every time HEAD moves. It's your undo history for Git itself.

Viewing the Reflog

Example: See recent HEAD movements

```bash
git reflog
```

```
e4f5a6b (HEAD -> main) HEAD@{0}: commit: Add greet function
a1b2c3d HEAD@{1}: commit (initial): Initial commit
f7g8h9i HEAD@{2}: checkout: moving from feature/login to main
b2c3d4e HEAD@{3}: commit: Add login page
a1b2c3d HEAD@{4}: checkout: moving from main to feature/login
```

What happened?

Every checkout, commit, rebase, reset, merge, and cherry-pick is logged here with a timestamp.

Example: See reflog with dates

```bash
git reflog --date=relative
```

```
e4f5a6b HEAD@{5 minutes ago}: commit: Add greet function
a1b2c3d HEAD@{10 minutes ago}: commit (initial): Initial commit
```

Recovering from Mistakes

Example: Undo an accidental hard reset

```bash
# Oops, I reset too far
git reset --hard HEAD~3

# Find the commit I lost
git reflog
```

```
a1b2c3d (HEAD -> main) HEAD@{0}: reset: moving to HEAD~3
e4f5a6b HEAD@{1}: commit: Add greet function
d4e5f6g HEAD@{2}: commit: Add routes
c3d4e5f HEAD@{3}: commit: Add controller
```

```bash
# Recover!
git reset --hard e4f5a6b
```

What happened?

The reflog still remembered where HEAD was before the reset. You can always go back.

Example: Recover a deleted branch

```bash
git branch -D feature/experiment
# Oh no, I needed that!

git reflog | grep experiment
```

```
b2c3d4e HEAD@{7}: checkout: moving from feature/experiment to main
```

```bash
git branch feature/experiment b2c3d4e
```

The branch is back.

Example: Recover after a bad rebase

```bash
# Find where the branch was before the rebase
git reflog
```

```
f6g7h8i HEAD@{0}: rebase (finish): returning to refs/heads/feature/signup
f6g7h8i HEAD@{1}: rebase (pick): Add signup form
e4f5a6b HEAD@{2}: rebase (start): checkout main
b2c3d4e HEAD@{3}: commit: Add signup form    ← this is the pre-rebase state
```

```bash
git reset --hard b2c3d4e
```

Reflog Expiration

Reflog entries expire after **90 days** by default (30 days for unreachable commits). You can configure this:

```bash
git config --global gc.reflogExpire 180
```

Branch Reflog

Each branch has its own reflog too.

Example: See reflog for a specific branch

```bash
git reflog show feature/login
```

Chapter 15

Bisect: Finding Bugs

`git bisect` uses binary search to find the exact commit that introduced a bug.

How It Works

You tell Git one "good" commit (where the bug didn't exist) and one "bad" commit (where the bug exists). Git checks out the midpoint and asks: good or bad? It keeps halving until it finds the exact commit.

With 1,000 commits between good and bad, bisect finds the culprit in about 10 steps.

Manual Bisect

Example: Find the commit that broke the tests

```bash
# Start bisecting
git bisect start

# Current commit is broken
git bisect bad

# This older commit was fine
git bisect good v1.0.0
```

```
Bisecting: 15 revisions left to test after this (roughly 4 steps)
[c3d4e5f] Add caching layer
```

Git checks out a commit in the middle. Test it:

```bash
# Run your tests
npm test

# If tests pass:
git bisect good

# If tests fail:
git bisect bad
```

Repeat until Git narrows it down:

```
b2c3d4e is the first bad commit
commit b2c3d4e
Author: Someone <someone@example.com>
Date:   Mon Mar 10 2025

    Refactor database queries
```

Example: End the bisect session

```bash
git bisect reset
```

This returns you to the branch you were on before bisecting.

Automated Bisect

If you have a script or test that can determine good/bad automatically, let Git run it for you.

Example: Bisect with a test script

```bash
git bisect start HEAD v1.0.0
git bisect run npm test
```

```
running 'npm test'...
...
b2c3d4e is the first bad commit
```

Example: Bisect with a custom script

```bash
git bisect start HEAD v1.0.0
git bisect run ./test-bug.sh
```

The script must exit with code `0` for good and `1` (or `125` to skip) for bad.

Example: test-bug.sh

```bash
#!/bin/bash
# Check if the specific bug exists
grep -q "broken_function" app.js && exit 1 || exit 0
```

Skipping a Commit

Sometimes a commit can't be tested (doesn't compile, for example):

```bash
git bisect skip
```

Git will try a nearby commit instead.

Viewing Bisect Log

Example: See the bisect history

```bash
git bisect log
```

```
git bisect start
# bad: [e4f5a6b] HEAD
git bisect bad e4f5a6b
# good: [a1b2c3d] v1.0.0
git bisect good a1b2c3d
# good: [c3d4e5f] Add caching layer
git bisect good c3d4e5f
# bad: [d4e5f6g] Update dependencies
git bisect bad d4e5f6g
```

Chapter 16

Worktrees

Worktrees let you check out multiple branches at the same time, each in its own directory — without cloning the repo again.

Why Worktrees?

You're working on a feature and someone asks you to review a PR or fix a bug on `main`. Without worktrees, you'd have to stash or commit your work, switch branches, do the review, switch back. With worktrees, you just open a second directory.

Creating a Worktree

Example: Check out a branch in a new directory

```bash
git worktree add ../hotfix main
```

```
Preparing worktree (checking out 'main')
HEAD is now at e4f5a6b Add greet function
```

```bash
ls ../hotfix/
```

```
README.md  index.js  .gitignore
```

What happened?

Git created a new working directory at `../hotfix` linked to the same repository. Changes in either directory share the same `.git` database. The branch `main` is now checked out in `../hotfix`.

Example: Create a worktree with a new branch

```bash
git worktree add ../bugfix -b bugfix/issue-42
```

Listing Worktrees

Example: See all worktrees

```bash
git worktree list
```

```
/home/you/project         e4f5a6b [feature/signup]
/home/you/hotfix          e4f5a6b [main]
/home/you/bugfix          e4f5a6b [bugfix/issue-42]
```

Removing a Worktree

Example: Remove a worktree

```bash
git worktree remove ../hotfix
```

Or manually delete the directory and then clean up:

```bash
rm -rf ../hotfix
git worktree prune
```

Rules and Limitations

A branch can only be checked out in **one** worktree at a time. If you try:

```bash
git worktree add ../another main
```

```
fatal: 'main' is already checked out at '/home/you/hotfix'
```

This prevents conflicting edits to the same branch.

Chapter 17

Submodules

Submodules let you include one Git repository inside another, pinned to a specific commit.

When to Use Submodules

Common use cases: shared libraries, vendor dependencies, monorepo-like setups where components have independent release cycles.

Adding a Submodule

Example: Add a submodule

```bash
git submodule add https://github.com/example/shared-lib.git libs/shared
```

```
Cloning into '/home/you/project/libs/shared'...
done.
```

```bash
git status
```

```
Changes to be committed:
        new file:   .gitmodules
        new file:   libs/shared
```

Example: See the .gitmodules file

```bash
cat .gitmodules
```

```ini
[submodule "libs/shared"]
    path = libs/shared
    url = https://github.com/example/shared-lib.git
```

```bash
git commit -m "Add shared-lib submodule"
```

> **What happened?**
> Git recorded the submodule's URL and the exact commit it points to. The parent repo doesn't store the submodule's files — it stores a pointer.

Cloning a Repo with Submodules

Example: Clone and initialize submodules

```bash
git clone --recurse-submodules https://github.com/boringcollege/project.git
```

Or if you already cloned:

```bash
git submodule init
git submodule update
```

Or in one command:

```bash
git submodule update --init --recursive
```

Updating a Submodule

Example: Pull latest changes in a submodule

```bash
cd libs/shared
git fetch
git checkout v2.0.0
cd ../..
git add libs/shared
git commit -m "Update shared-lib to v2.0.0"
```

Example: Update all submodules to their latest remote commit

```bash
git submodule update --remote
```

> **What happened?**
>
> Each submodule is an independent repo. You update it like any repo, then commit the new pointer in the parent.

Checking Submodule Status

Example: See which commit each submodule is on

```bash
git submodule status
```

```
 a1b2c3d4e5f6 libs/shared (v2.0.0)
```

Removing a Submodule

There's no single command. Here are the steps:

```bash
# 1. Remove from .gitmodules
git config -f .gitmodules --remove-section submodule.libs/shared

# 2. Remove from .git/config
git config --remove-section submodule.libs/shared

# 3. Remove the tracked directory
git rm --cached libs/shared

# 4. Remove the submodule directory
rm -rf libs/shared

# 5. Remove the submodule's .git data
rm -rf .git/modules/libs/shared

# 6. Commit
git commit -m "Remove shared-lib submodule"
```

Common Pitfalls

Detached HEAD: Submodules are always checked out in detached HEAD state. If you want to make changes, create a branch first:

```bash
cd libs/shared
git switch -c my-changes
```

Forgetting to push submodule changes: If you commit a new submodule pointer in the parent but forget to push the submodule's commits, collaborators will get errors. Always push the submodule first, then the parent.

```bash
cd libs/shared
git push
cd ../..
git push
```

Part V

Real-World Workflows

Chapter 18

Branching Strategies

A branching strategy is a convention your team agrees on for how branches are created, named, and merged.

Git Flow

The classic model. Good for software with scheduled releases.

```
main ──────────────────────────────────── (production releases)
               \                    /
release/1.0 ──────────────────── ── (stabilization)
                     \           /
develop ──────────────────────── (integration)
           \      /    \      /
feature/a   ────     ────  feature/b
```

Branches:

- `main` — always production-ready, tagged with version numbers.
- `develop` — integration branch for the next release.
- `feature/*` — branch off `develop`, merge back to `develop`.
- `release/*` — branch off `develop` for stabilization, merge to `main` and `develop`.
- `hotfix/*` — branch off `main` for urgent fixes, merge to `main` and `develop`.

Example: Git Flow workflow

BASH

```bash
# Start a feature
git switch develop
git switch -c feature/user-profile

# Work on the feature
git commit -m "Add profile page"
git commit -m "Add avatar upload"

# Merge back to develop
git switch develop
git merge --no-ff feature/user-profile
git branch -d feature/user-profile
```

GitHub Flow

Simpler. Good for teams that deploy continuously.

```
main ──────────────────── (always deployable)
        \      /    \      /
feature  ────     ────
         (PR + review)
```

Rules:

1. `main` is always deployable.
2. Branch off `main` for every change.
3. Open a pull request.
4. After review and CI passes, merge to `main`.
5. Deploy immediately.

Example: GitHub Flow workflow

```bash
# Start work
git switch -c feature/dark-mode
git commit -m "Add dark mode toggle"
git push -u origin feature/dark-mode

# Open a PR on GitHub, get reviews, then merge via the UI
# After merge:
git switch main
git pull
git branch -d feature/dark-mode
```

Trunk-Based Development

Even simpler. Everyone commits to `main` (the trunk) with short-lived branches.

```
main ──────────────────────────── (continuous)
        \  /       \  /
        ──         (short-lived, <1 day)
```

Rules:

1. Branches live for hours, not days.

2. Everyone integrates to `main` at least once a day.
3. Feature flags hide incomplete work.
4. CI/CD is mandatory.

Example: Trunk-based workflow

```bash
git switch -c fix/button-color
git commit -m "Fix button color on mobile"
git push -u origin fix/button-color
# PR reviewed and merged same day
```

Which Strategy Should You Use?

```
Team Size        Release Cadence       Strategy
_____________________________________________
1-3 people       Continuous            Trunk-based or GitHub Flow
3-10 people      Weekly/biweekly       GitHub Flow
10+ people       Scheduled releases    Git Flow
Any size         Regulated/audited     Git Flow (for traceability)
```

There's no universally correct answer. Pick the simplest strategy that works for your team and change it when it stops working.

Branch Naming Conventions

Whatever strategy you choose, use consistent names:

```
feature/user-authentication
bugfix/login-redirect
hotfix/security-patch
release/2.1.0
chore/update-dependencies
docs/api-reference
```

Chapter 19

Writing Good Commit Messages

Good commit messages make your project's history useful. Bad ones make it noise.

The Format

```
<type>: <short summary> (50 chars or less)

<optional body — wrap at 72 chars>
Explain WHAT changed and WHY, not HOW.

<optional footer>
Refs: #123
Co-authored-by: Jane Doe <jane@example.com>
```

The Seven Rules

1. Separate subject from body with a blank line.
2. Limit the subject line to 50 characters.
3. Capitalize the subject line.
4. Do not end the subject line with a period.
5. Use the imperative mood ("Add feature" not "Added feature").
6. Wrap the body at 72 characters.
7. Use the body to explain *what* and *why*, not *how*.

Example: Good commit messages

```
Add rate limiting to login endpoint

Without rate limiting, the login endpoint was vulnerable to brute-force
attacks. This adds a sliding window rate limiter that allows 5 attempts
per minute per IP address.

Refs: #247
```

```
Fix crash when user has no profile photo

The avatar component assumed a non-null photo URL. Users who never
uploaded a photo triggered a TypeError. This adds a fallback to a
default avatar.
```

```
Upgrade React from 18.2 to 19.0

Breaking changes addressed:
- Replaced legacy context API usage in ThemeProvider
- Updated all class components to function components
- Removed deprecated lifecycle methods

See: https://react.dev/blog/2024/12/05/react-19
```

Example: Bad commit messages

```
fix bug                         ← What bug? Where?
update                          ← Update what?
WIP                             ← Don't commit WIP to shared branches
changes                         ← Meaningless
fix review comments             ← Which comments? What changed?
asdfasdf                        ← Come on
```

Conventional Commits

A widely adopted convention that adds a type prefix:

```
feat: add user registration
fix: prevent crash on empty input
docs: update API reference
style: format code with prettier
refactor: extract validation logic
test: add tests for login flow
chore: update dependencies
perf: cache database queries
ci: add GitHub Actions workflow
build: switch from webpack to vite
```

Example: With scope

```
feat(auth): add two-factor authentication
fix(api): handle timeout errors gracefully
docs(readme): add installation instructions
```

Example: Breaking change

```
feat(api)!: change response format to JSON:API

BREAKING CHANGE: All API responses now follow the JSON:API spec.
The old format is no longer supported. See migration guide at
docs/migration-v3.md
```

The Imperative Mood Test

Your subject line should complete this sentence:

> *"If applied, this commit will ___**your subject line here**___."*

- "If applied, this commit will **add rate limiting to login endpoint**."
- "If applied, this commit will **fix crash when user has no profile photo**."
- "If applied, this commit will **fixed a bug**."
- "If applied, this commit will **some changes**."

Commit Atomicity

Each commit should be one logical change. If you can describe a commit as "X **and** Y," it should probably be two commits.

```bash
# Bad: one commit doing two things
git commit -m "Fix login bug and add dark mode"

# Good: two focused commits
git commit -m "Fix login redirect after session timeout"
git commit -m "Add dark mode toggle to settings page"
```

Use `git add -p` (see Chapter 2) to stage parts of a file when you've made multiple unrelated changes.

Chapter 20

Git Hooks

Git hooks are scripts that run automatically at specific points in the Git workflow. They live in `.git/hooks/`.

Available Hooks

```
Hook                  When It Runs
──────────────────────────────────────────────────
pre-commit            Before a commit is created
prepare-commit-msg    After default message, before editor opens
commit-msg            After you write the message (validate it)
post-commit           After a commit is created
pre-push              Before a push
pre-rebase            Before a rebase
post-merge            After a merge
post-checkout         After checkout / switch
```

Your First Hook

Example: pre-commit hook that runs linting

BASH

```bash
cat > .git/hooks/pre-commit << 'EOF'
#!/bin/bash
echo "Running linter..."
npm run lint

if [ $? -ne 0 ]; then
    echo "X Lint failed. Fix errors before committing."
    exit 1
fi

echo " Lint passed."
EOF

chmod +x .git/hooks/pre-commit
```

Now try committing:

```bash
git commit -m "Test hook"
```

```
Running linter...
 Lint passed.
[main a1b2c3d] Test hook
```

> **What happened?**
>
> The hook ran before the commit. If it exits with a non-zero code, the commit is aborted.

Example: commit-msg hook to enforce format

```bash
cat > .git/hooks/commit-msg << 'EOF'
#!/bin/bash
MSG=$(cat "$1")
PATTERN="^(feat|fix|docs|style|refactor|test|chore|perf|ci|build)(\(.+\))?: .{1,50}"

if ! echo "$MSG" | grep -qE "$PATTERN"; then
    echo "X Commit message does not follow Conventional Commits format."
    echo "   Expected: <type>(optional scope): <description>"
    echo "   Example:  feat(auth): add login endpoint"
    exit 1
fi
EOF

chmod +x .git/hooks/commit-msg
```

```bash
git commit -m "stuff"
```

```
X Commit message does not follow Conventional Commits format.
   Expected: <type>(optional scope): <description>
   Example:  feat(auth): add login endpoint
```

```bash
git commit -m "feat: add user model"
```

```
[main b2c3d4e] feat: add user model
```

Example: pre-push hook to run tests

```bash
cat > .git/hooks/pre-push << 'EOF'
#!/bin/bash
echo "Running tests before push..."
npm test

if [ $? -ne 0 ]; then
    echo "X Tests failed. Push aborted."
    exit 1
fi
EOF

chmod +x .git/hooks/pre-push
```

The Problem: Hooks Aren't Shared

`.git/hooks/` is not tracked by Git, so hooks don't travel with the repository. Your teammates won't have them.

Solution 1: Use a hooks directory in the repo

```bash
mkdir .githooks

# Move hooks there
mv .git/hooks/pre-commit .githooks/

# Tell Git to use this directory
git config core.hooksPath .githooks

# Commit the hooks
git add .githooks/
git commit -m "chore: add shared git hooks"
```

Solution 2: Use a tool like Husky (Node.js projects)

```bash
npm install --save-dev husky
npx husky init
```

```bash
# Create a pre-commit hook
echo "npm run lint" > .husky/pre-commit
```

Now every developer who runs `npm install` gets the hooks automatically.

Bypassing Hooks

Example: Skip hooks for a single commit

```bash
git commit --no-verify -m "WIP: quick save"
```

Example: Skip hooks for a push

```bash
git push --no-verify
```

> **Warning**
> Use sparingly. If you're bypassing hooks often, the hooks might be too strict or too slow.

Chapter 21

Tips & Tricks

A collection of useful Git techniques that don't fit neatly into other chapters.

Aliases

Example: Set up useful aliases

```bash
git config --global alias.st "status -sb"
git config --global alias.co "checkout"
git config --global alias.br "branch"
git config --global alias.ci "commit"
git config --global alias.lg "log --oneline --graph --all --decorate"
git config --global alias.last "log -1 HEAD --stat"
git config --global alias.unstage "restore --staged"
git config --global alias.amend "commit --amend --no-edit"
```

Now you can type:

```bash
git st        # instead of git status -sb
git lg        # beautiful graph log
git unstage file.js
```

Ignoring Files Locally

Example: Ignore a file without changing .gitignore

```bash
echo "my-local-notes.txt" >> .git/info/exclude
```

What happened?

`.git/info/exclude` works like `.gitignore` but isn't tracked, so it's private to your machine.

Example: Temporarily ignore a tracked file

```bash
git update-index --assume-unchanged config/local.yml
```

Undo it:

```bash
git update-index --no-assume-unchanged config/local.yml
```

Finding Things

Example: Find which commit deleted a file

```bash
git log --diff-filter=D --summary -- path/to/deleted-file.js
```

Example: Find all commits by a specific author

```bash
git log --author="Dariush" --oneline
```

Example: Find all commits touching a specific function

```bash
git log -p -S "calculateTotal" -- "*.js"
```

Example: Find all TODO comments added recently

```bash
git diff HEAD~10..HEAD -S "TODO"
```

Cleaning Up

Example: Remove all merged branches

```bash
git branch --merged main | grep -v "main" | xargs git branch -d
```

Example: Garbage collect

```bash
git gc --prune=now
```

This cleans up unnecessary files and optimizes the repository.

Working with Large Files

Example: See the largest files in history

```bash
git rev-list --objects --all \
  | git cat-file --batch-check='%(objecttype) %(objectname) %(objectsize) %(rest)' \
  | awk '/^blob/ {print $3, $4}' \
  | sort -rn \
  | head -10
```

Example: Set up Git LFS for large files

```bash
git lfs install
git lfs track "*.psd"
git lfs track "*.mp4"
git add .gitattributes
git commit -m "Track large files with Git LFS"
```

Useful One-Liners

Example: Count commits per author

```bash
git shortlog -sn --all
```

Example: See which files change most often

```bash
git log --pretty=format: --name-only | sort | uniq -c | sort -rn | head -10
```

Example: See total lines added/removed by author

```bash
git log --author="Dariush" --pretty=tformat: --numstat \
  | awk '{ add += $1; del += $2 } END { printf "Added: %s Removed: %s\n", add, del }'
```

Example: Create an archive of the current HEAD

```bash
git archive --format=tar.gz --prefix=project-v1.0/ HEAD > project-v1.0.tar.gz
```

Example: Show a file as it was 5 commits ago

```bash
git show HEAD~5:path/to/file.js
```

Sparse Checkout

When a repo is huge and you only need a few directories:

Example: Clone only specific directories

```bash
git clone --filter=blob:none --sparse https://github.com/big/repo.git
cd repo
git sparse-checkout set src/frontend docs
```

What happened?

Git cloned the repo's metadata but only downloaded files in `src/frontend` and `docs`. Everything else is downloaded on demand.

Git Maintenance

For large repositories, enable background maintenance:

```bash
git maintenance start
```

This runs hourly tasks like prefetching, garbage collection, and commit-graph updates.

Appendices

Git Configuration Cheatsheet

Configuration Levels

```bash
git config --system   # /etc/gitconfig       (all users)
git config --global   # ~/.gitconfig          (your user)
git config --local    # .git/config           (this repo only)
```

Local overrides global, which overrides system.

Identity

```bash
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Editor

```bash
git config --global core.editor "vim"
git config --global core.editor "code --wait"       # VS Code
git config --global core.editor "nano"
git config --global core.editor "subl -n -w"        # Sublime Text
```

Default Branch

```bash
git config --global init.defaultBranch main
```

Line Endings

```bash
# On macOS/Linux
git config --global core.autocrlf input

# On Windows
git config --global core.autocrlf true
```

Merge & Diff Tools

```bash
git config --global merge.tool vimdiff
git config --global diff.tool vscode
git config --global difftool.vscode.cmd 'code --wait --diff $LOCAL $REMOTE'
```

Pull Behavior

```bash
# Rebase by default (cleaner history)
git config --global pull.rebase true

# Only allow fast-forward pulls
git config --global pull.ff only
```

Push Behavior

```bash
# Push only the current branch
git config --global push.default current

# Auto set upstream when pushing a new branch
git config --global push.autoSetupRemote true
```

Colors

```bash
git config --global color.ui auto
```

Credential Caching

```bash
# Cache credentials in memory for 1 hour
git config --global credential.helper 'cache --timeout=3600'

# On macOS, use Keychain
git config --global credential.helper osxkeychain

# On Windows, use Credential Manager
git config --global credential.helper manager
```

Useful Misc Settings

```bash
# Show untracked files in subdirectories
git config --global status.showUntrackedFiles all

# Enable rerere (reuse recorded resolution)
git config --global rerere.enabled true

# Auto-correct typos (with 0.5s delay)
git config --global help.autocorrect 5

# Use histogram diff algorithm (better output)
git config --global diff.algorithm histogram

# Sign commits with GPG
git config --global commit.gpgsign true
git config --global user.signingkey YOUR_GPG_KEY_ID
```

View All Settings

```bash
git config --list --show-origin
```

This shows every setting and which file it comes from.

.gitignore Patterns

How .gitignore Works

Each line in `.gitignore` is a pattern. Git ignores matching files.

```
# This is a comment
*.log           # Ignore all .log files
!important.log  # But NOT this one
/build          # Ignore build/ in root only
build/          # Ignore any directory named build
**/temp         # Ignore temp anywhere in the tree
doc/**/*.pdf    # Ignore PDFs in doc/ subdirectories
```

Pattern Syntax

```
Pattern            Matches
──────────────────────────────────────────
*.ext              Any file with that extension
dir/               A directory (and its contents)
/dir               Only in the repo root
**/pattern         At any depth
pattern/**         Everything inside pattern/
!pattern           Negate (un-ignore) a previous rule
?                  Any single character
[abc]              Any character in the set
[0-9]              Any digit
```

Common .gitignore Templates

Node.js

```gitignore
node_modules/
dist/
build/
.env
.env.local
*.log
npm-debug.log*
.DS_Store
coverage/
```

Python

```gitignore
__pycache__/
*.py[cod]
*.egg-info/
dist/
build/
.venv/
venv/
.env
*.egg
.pytest_cache/
.mypy_cache/
```

Java

```gitignore
*.class
*.jar
*.war
target/
.gradle/
build/
.idea/
*.iml
```

Go

```gitignore
bin/
vendor/
*.exe
*.test
*.out
```

Rust

```gitignore
target/
Cargo.lock  # only for libraries, keep for binaries
```

General / IDE

```gitignore
# macOS
.DS_Store
.AppleDouble
.LSOverride

# Windows
Thumbs.db
desktop.ini

# Linux
*~
.nfs*

# IDEs
.idea/
.vscode/
*.swp
*.swo
*~
.project
.classpath
.settings/

# Environment
.env
.env.local
.env.*.local
```

Global .gitignore

Set a global ignore file for patterns that apply to all your repos:

```bash
git config --global core.excludesfile ~/.gitignore_global
```

```bash
# ~/.gitignore_global
.DS_Store
Thumbs.db
*.swp
.idea/
.vscode/
```

Ignoring Already-Tracked Files

`.gitignore` only affects untracked files. To ignore a file that's already tracked:

```bash
git rm --cached filename
echo "filename" >> .gitignore
git commit -m "Stop tracking filename"
```

Debugging .gitignore

Example: Check why a file is ignored

```bash
git check-ignore -v path/to/file
```

```
.gitignore:3:*.log    debug.log
```

This tells you which rule in which file is causing the ignore.

Resources

The github/gitignore repository has comprehensive `.gitignore` templates for virtually every language and framework.

Aliases Worth Having

Copy and paste the following into your terminal to set up all aliases at once.

Essential Aliases

```bash
# Short forms
git config --global alias.st "status -sb"
git config --global alias.co "checkout"
git config --global alias.sw "switch"
git config --global alias.br "branch"
git config --global alias.ci "commit"
git config --global alias.cp "cherry-pick"
```

Log Aliases

```bash
# Beautiful one-line graph log
git config --global alias.lg "log --oneline --graph --all --decorate"

# Detailed graph log
git config --global alias.ll "log --graph --all --decorate --format='%C(yellow)%h%C(reset) %C(cya…

# Show the last commit
git config --global alias.last "log -1 HEAD --stat"

# Commits from today
git config --global alias.today "log --since=midnight --oneline --author='$(git config user.name)'

# Commits from this week
git config --global alias.week "log --since='1 week ago' --oneline --author='$(git config user.na…
```

Undo Aliases

```bash
# Unstage a file
git config --global alias.unstage "restore --staged"

# Amend the last commit without editing the message
git config --global alias.amend "commit --amend --no-edit"

# Undo the last commit, keep changes staged
git config --global alias.undo "reset --soft HEAD~1"

# Discard all uncommitted changes (dangerous!)
git config --global alias.nuke "!git reset --hard HEAD && git clean -fd"
```

Branch Aliases

```bash
# Delete all merged branches
git config --global alias.cleanup "!git branch --merged main | grep -v 'main' | xargs -r git bran…

# Show branches sorted by last commit date
git config --global alias.recent "branch --sort=-committerdate --format='%(committerdate:relative…

# Show current branch name
git config --global alias.current "branch --show-current"
```

Diff Aliases

```bash
# Word-level diff (useful for prose)
git config --global alias.wdiff "diff --word-diff"

# Show only filenames that changed
git config --global alias.changed "diff --name-only"

# Diff of staged changes
git config --global alias.staged "diff --staged"
```

Workflow Aliases

```bash
# Save work in progress
git config --global alias.wip "!git add -A && git commit -m 'WIP: work in progress [skip ci]'"

# Quick save and push
git config --global alias.save "!git add -A && git commit -m 'chore: savepoint' && git push"

# Pull with rebase
git config --global alias.up "pull --rebase --autostash"

# Force push safely
git config --global alias.pushf "push --force-with-lease"
```

Info Aliases

```bash
# Who contributed?
git config --global alias.who "shortlog -sn --all"

# List all aliases
git config --global alias.aliases "config --get-regexp ^alias\\."

# Show repo root directory
git config --global alias.root "rev-parse --show-toplevel"
```

Usage Examples

After setting up these aliases:

```bash
$ git st
## main...origin/main
 M README.md

$ git lg
* e4f5a6b (HEAD -> main) Add greet function
* a1b2c3d Initial commit

$ git recent
5 minutes ago    main
2 hours ago      feature/login
3 days ago       feature/old-experiment

$ git today
e4f5a6b Add greet function
d4e5f6g Fix tests
c3d4e5f Update README

$ git undo
# Last commit undone, changes are staged and ready to re-commit
```

Shell Aliases (bonus)

Add these to your `~/.bashrc` or `~/.zshrc` for even shorter commands:

```bash
alias g="git"
alias gs="git status -sb"
alias gc="git commit"
alias gp="git push"
alias gl="git pull"
alias gd="git diff"
alias gco="git checkout"
alias gsw="git switch"
alias gb="git branch"
alias ga="git add"
alias glg="git log --oneline --graph --all --decorate"
```

Now `gs` shows status, `glg` shows the graph, and `ga .` stages everything.